DATA 622 Homework 7: Neural Networks

Sabina Baraili

Table of contents

1 Overview **1**

2 Setup **2**

3 Data Setup **2**

4 Model Plan **3**

5 Problem 1: Assessing the Original Fit **5**
5.1 1(a) Baseline CNN 5
5.2 1(b) Class-Level Validation Accuracy 6
5.3 1(c) Misclassified Validation Images 8

6 Problem 2: Augmenting the Data **8**
6.1 2(a) Training With Augmentation 8

7 Problem 3: Widening the Network **11**
7.1 3(a) Wider CNN 11

8 Problem 4: Deep Learning **13**
8.1 4(a) Deeper CNN Without Batch Normalization 13
8.2 4(b) Deeper CNN With Batch Normalization 13
8.3 4(c) Final Test Set Comparison 15

9 Final Summary **16**

10 Reproducibility Note **17**

1 Overview

This report follows the Homework 7 questions in order. The goal is to use the CIFAR100 image data to evaluate a baseline convolutional neural network, then improve it with data augmentation, wider convolutional layers, deeper convolutional blocks, and batch normalization.

I use an 80/20 split of the official CIFAR100 training set for training and validation. The official test set is reserved for final model comparison only. This avoids using the test set as a tuning target.

The local machine used to prepare this PDF does not have a CUDA-enabled GPU, so the neural-network training runs were executed in Google Colab using an A100 GPU runtime. The final figures below use the observed Colab training histories and test-set results.

2 Setup

This report follows the lab questions in order and answers each one directly using CIFAR100. The code is included in the QMD source and hidden from the PDF.

3 Data Setup

CIFAR100 contains 60,000 color images across 100 classes. The images are small, 32 by 32 pixels, so fine-grained categories can be visually ambiguous.

The analysis uses 40,000 images for training, 10,000 for validation, and 10,000 for final testing. The validation set is used to choose the best epoch and compare model variants; the test set is held back until the end.

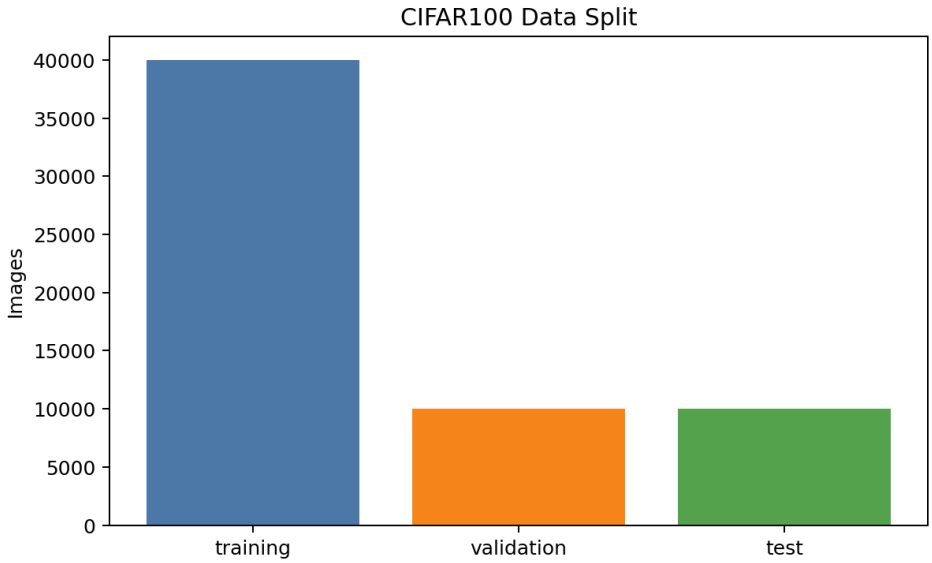

Figure 1: Train, validation, and test split used for CIFAR100.

Figure 2: Example CIFAR100 training images.

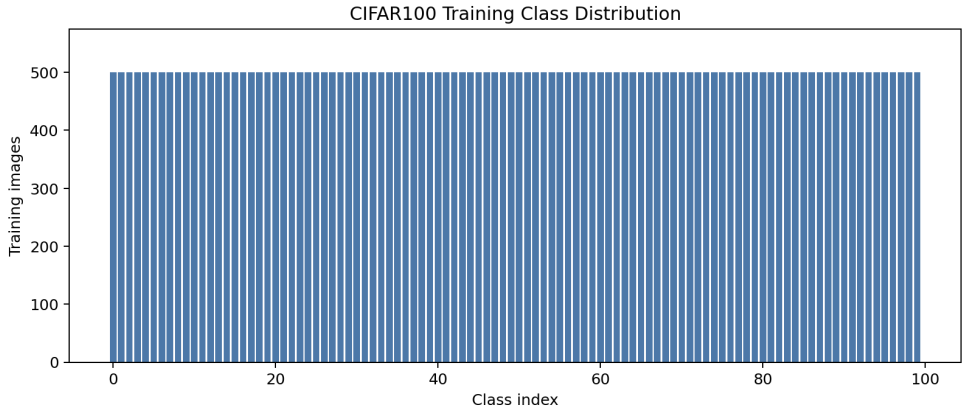

Figure 3: The CIFAR100 training set is balanced across classes.

Interpretation: The balanced class structure means accuracy is a reasonable first metric because each class contributes equally in the raw dataset. The small image size still makes the task difficult because many labels require fine visual distinctions.

4 Model Plan

The assignment asks for a sequence of models that gradually changes the amount of regularization, width, depth, and normalization.

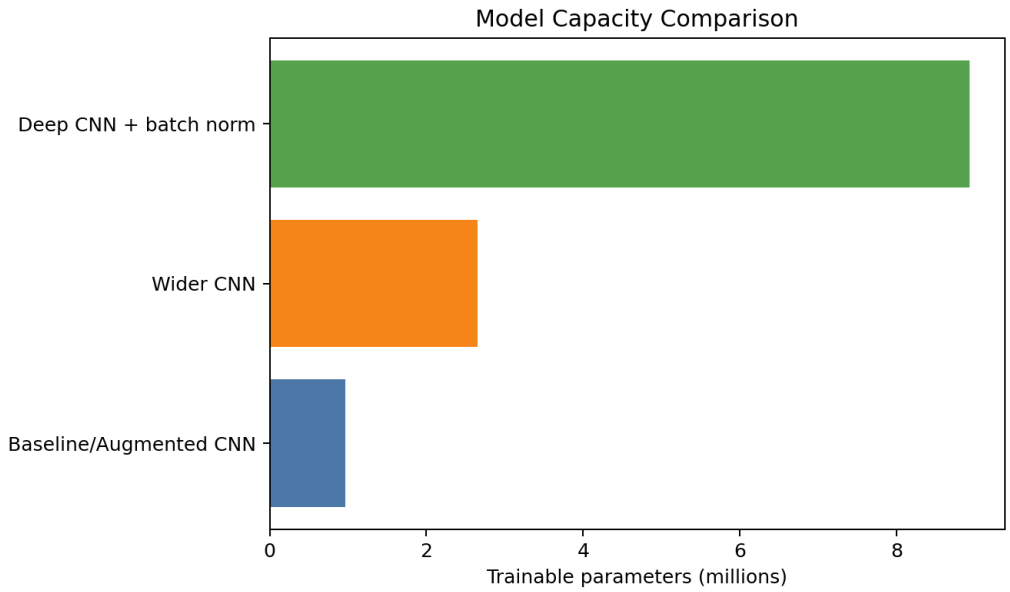

Figure 4: Model capacity comparison.

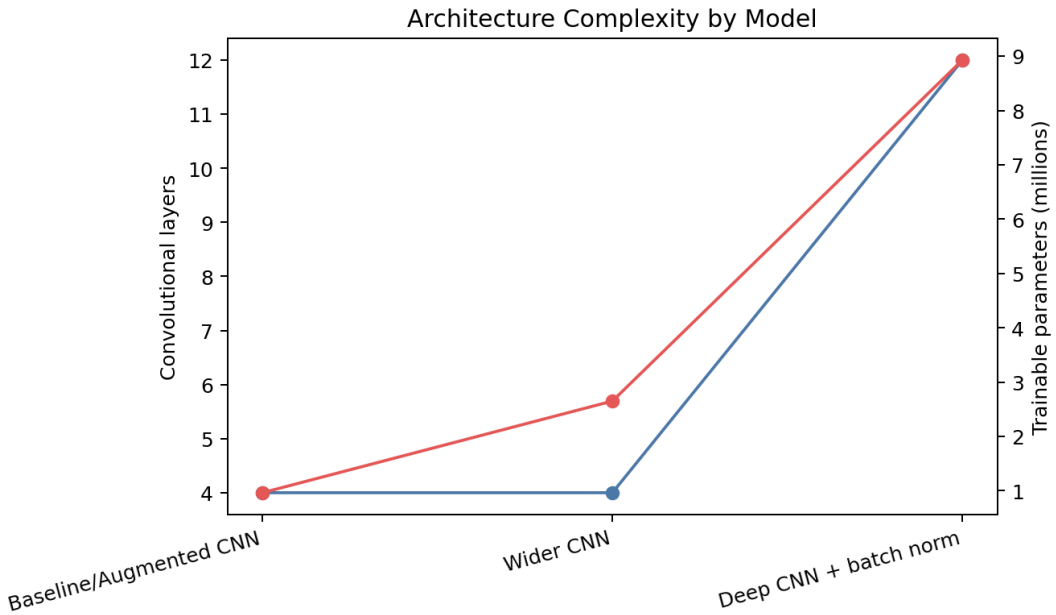

Figure 5: Architecture complexity increases most in Problems 3 and 4.

Interpretation: The baseline and augmented models have the same architecture, so any improvement in Problem 2 should come from better regularization through augmentation rather than more parameters. The wider and deeper models have much more capacity, which can improve representation but also raises the risk of overfitting.

5 Problem 1: Assessing the Original Fit

5.1 1(a) Baseline CNN

The baseline model uses four convolution/max-pooling blocks. Each block halves spatial resolution and increases channels from 3 input color channels to 32, 64, 128, and 256. The classifier then flattens the final feature map, applies dropout, uses a 512-unit ReLU layer, and outputs 100 logits.

The baseline is trained for 50 epochs with RMSprop and learning rate 0.001. The selected model is the epoch with the highest validation accuracy. The test set is evaluated only after that validation-based selection.

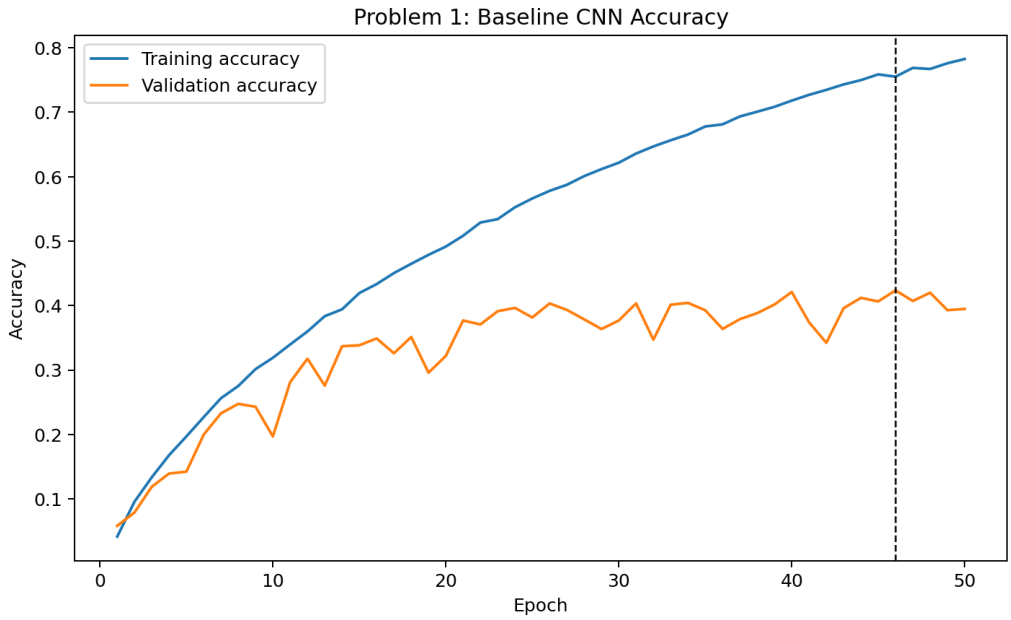

Figure 6: Baseline CNN training and validation accuracy.

Interpretation: The baseline reached its best validation accuracy at epoch 46, with validation accuracy 0.4236. The corresponding held-out test accuracy was 0.4316, which I save for the final comparison rather than using to tune the model. Training accuracy continued to increase across the run, while validation accuracy became less stable, so selecting the best validation epoch is preferable to using the final epoch.

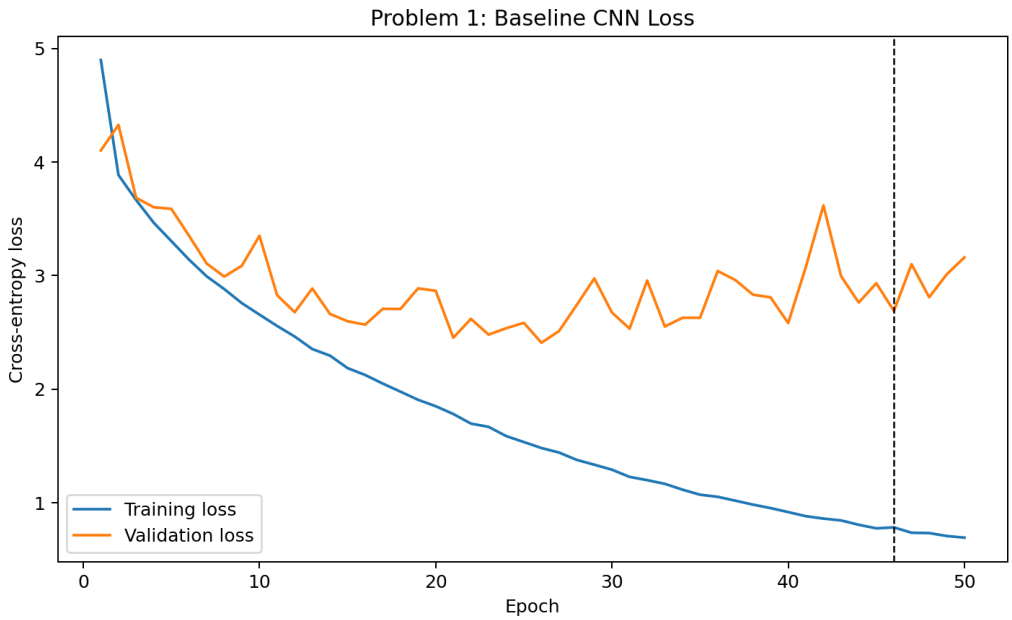

Figure 7: Baseline CNN training and validation loss.

Loss diagnostic: The loss curve gives a clearer view of overfitting than accuracy alone. The training loss keeps falling, while validation loss rises and fluctuates after the model has already found its best validation accuracy. This means the later epochs are fitting the training data more tightly without improving generalization.

5.2 1(b) Class-Level Validation Accuracy

After fitting the baseline model, I evaluate how performance varies by CIFAR100 class. Class-level accuracy is computed on the validation set by grouping predictions by true class label.

The validation predictions are grouped by true class label, then sorted to identify the 10 highest-accuracy classes and 10 lowest-accuracy classes. The QMD source contains the code for producing horizontal bar charts because that visual form is easier to compare than a long table of class names and decimals.

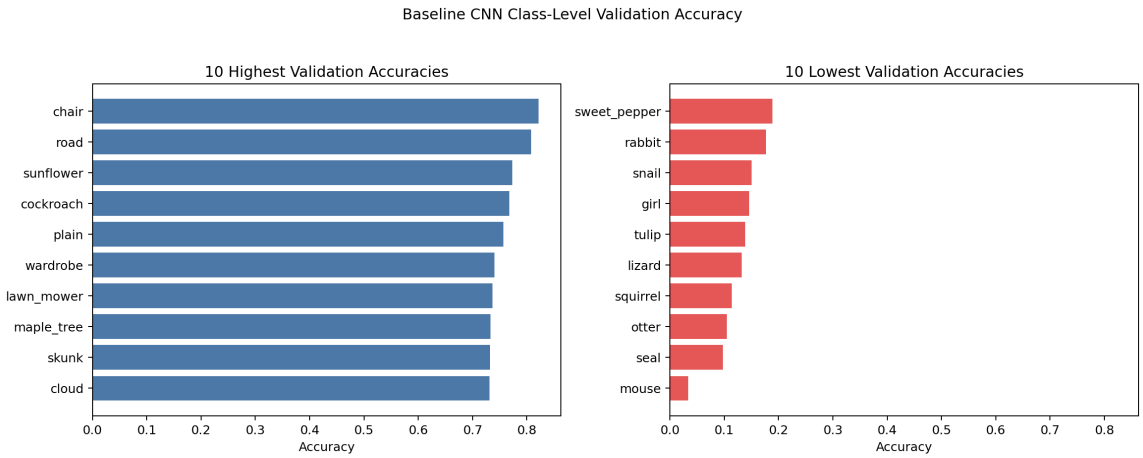

Figure 8: Baseline CNN classes with the highest and lowest validation accuracy.

Interpretation: Class-level plots are important because overall accuracy can hide uneven class performance. The strongest classes include visually distinctive labels such as chair, road, sunflower, and cockroach, each above 76% validation accuracy. The weakest classes include mouse, seal, otter, squirrel, and lizard, all below about 14% validation accuracy, which shows that the baseline struggles most on small or visually similar animal categories.

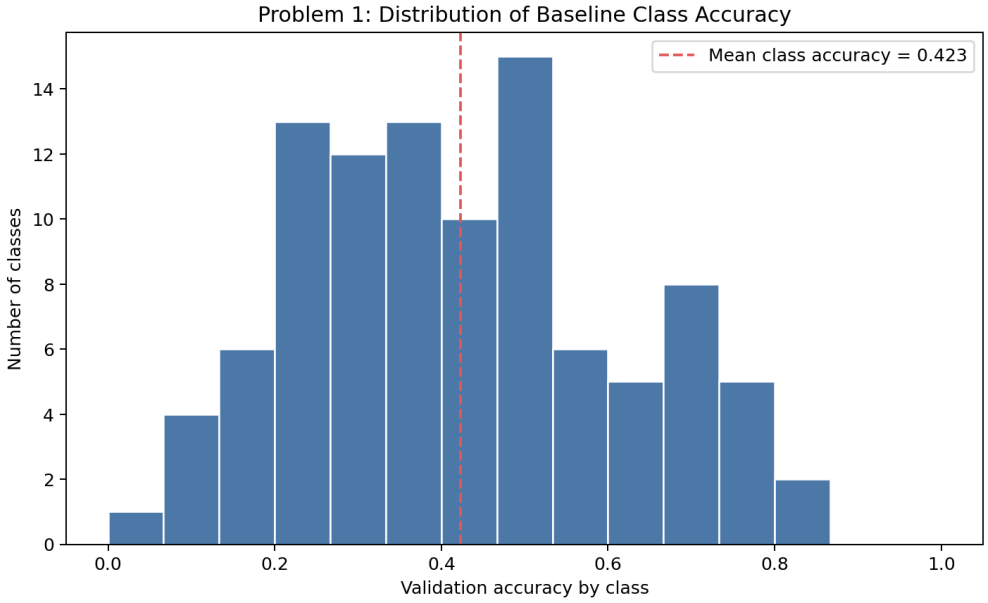

Figure 9: Distribution of baseline validation accuracy across the 100 CIFAR100 classes.

Additional diagnostic: The class-accuracy distribution shows that the baseline does not fail uniformly. Some classes are learned reasonably well, but many classes remain below the overall average. This is expected for CIFAR100 because each image is only 32 by 32 pixels and several labels are visually close.

5.3 1(c) Misclassified Validation Images

The baseline diagnostic uses a five-image panel of validation mistakes. Each panel displays the image, true label, and predicted label. This code is included for the Colab notebook workflow because it requires the trained baseline model object.

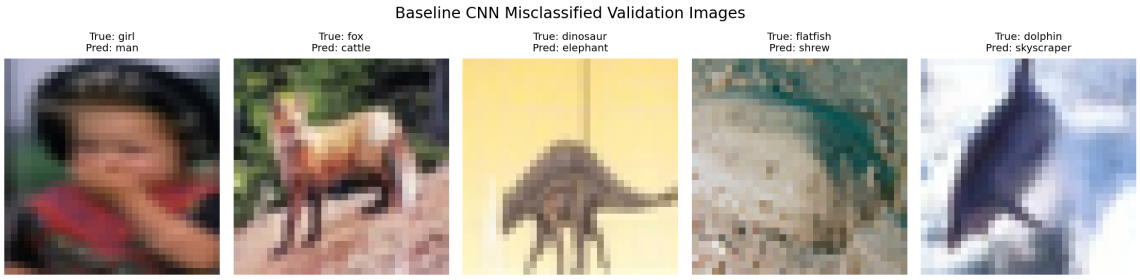

Figure 10: Five baseline validation images that were misclassified.

Interpretation: The example mistakes show both understandable confusions and unstable predictions from the baseline model. Some errors involve visually related labels, such as girl predicted as man, while others are much less plausible, such as dolphin predicted as skyscraper. This reinforces the class-level result: the baseline has learned useful features, but its representation is still too weak for many CIFAR100 classes.

6 Problem 2: Augmenting the Data

6.1 2(a) Training With Augmentation

I use modest augmentation on the training loader only: horizontal flips, padded random crops, mild color jitter, small rotations, and random erasing. Normalization is applied before random erasing, as required.

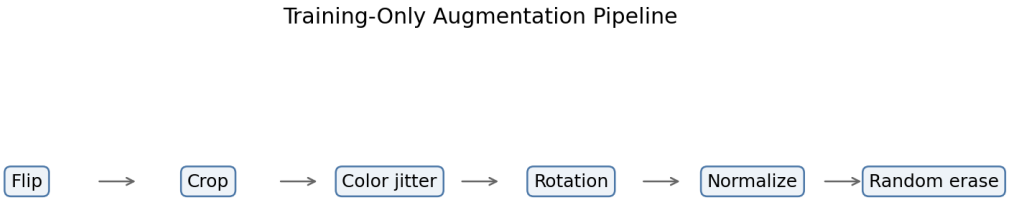

Figure 11: Training-only data augmentation pipeline.

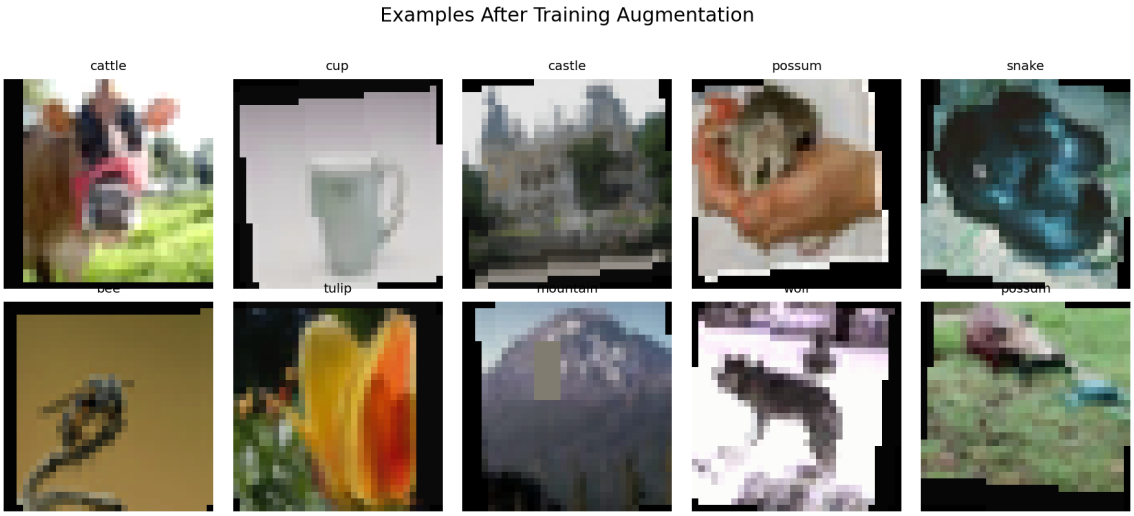

Figure 12: Examples of modest training augmentations.

The augmented model was trained for 40 epochs in the Colab run. I used the shorter run because Colab runtime limits made the full 300-epoch version impractical, and the assignment says to increase epochs up to 300 rather than requiring exactly 300.

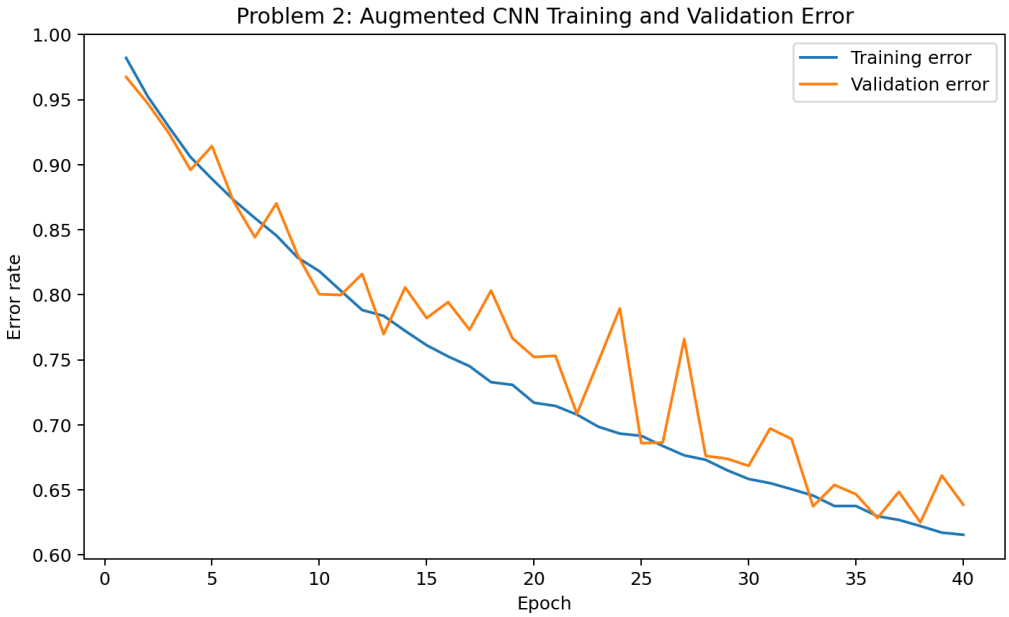

Figure 13: Training and validation error for the augmented CNN.

Figure 14: Validation error for the baseline and augmented CNN.

Interpretation: In this shortened run, augmentation did not improve final test accuracy; the augmented model reached 0.3730 test accuracy, or 0.6270 test error, compared with 0.4316 test accuracy and 0.5684 test error for the baseline. The likely reason is that augmentation makes the optimization problem harder, and the shorter training schedule was not long enough for the augmented model to catch up.

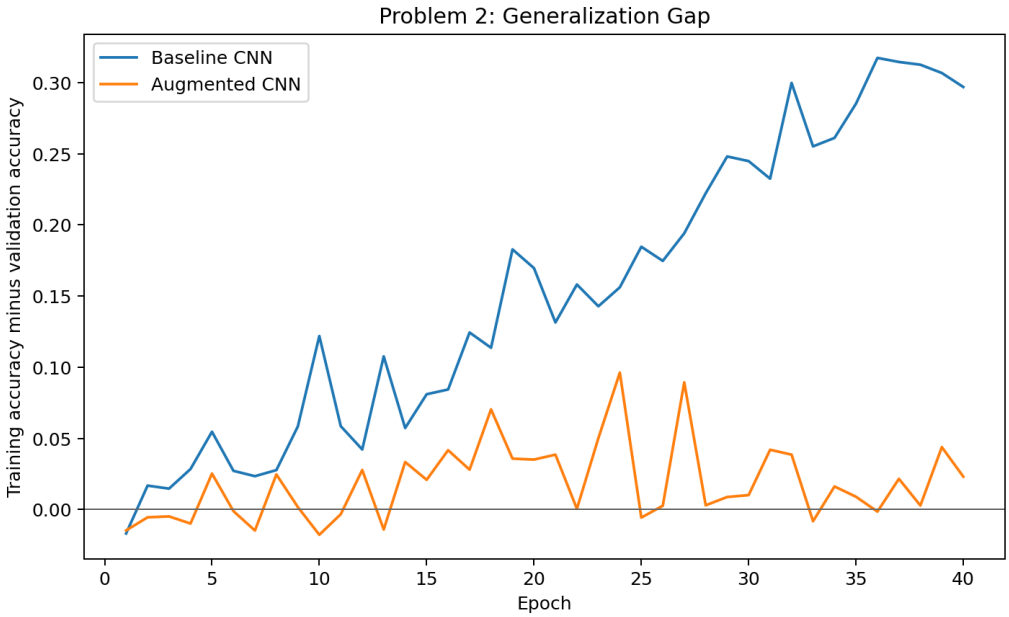

Figure 15: Generalization gap for the baseline and augmented CNNs.

Overfitting comparison: The gap plot compares training accuracy minus validation accuracy. The augmented model has a smaller gap, so augmentation did act as regularization. However, lower overfitting did not translate into higher test accuracy in this run because the augmented model also had lower training accuracy.

7 Problem 3: Widening the Network

7.1 3(a) Wider CNN

The wider network increases channels from 32-64-128-256 to 64-128-256-512. I use Adam, a lower learning rate, dropout, and weight decay because the wider model has more capacity.

The model capacity plot in the Model Plan section shows the main change: the wider model has about 2.65 million trainable parameters, compared with about 0.96 million for the augmented baseline architecture. The main diagnostic for this section is an overlay of augmented-model and wider-model validation accuracy across epochs.

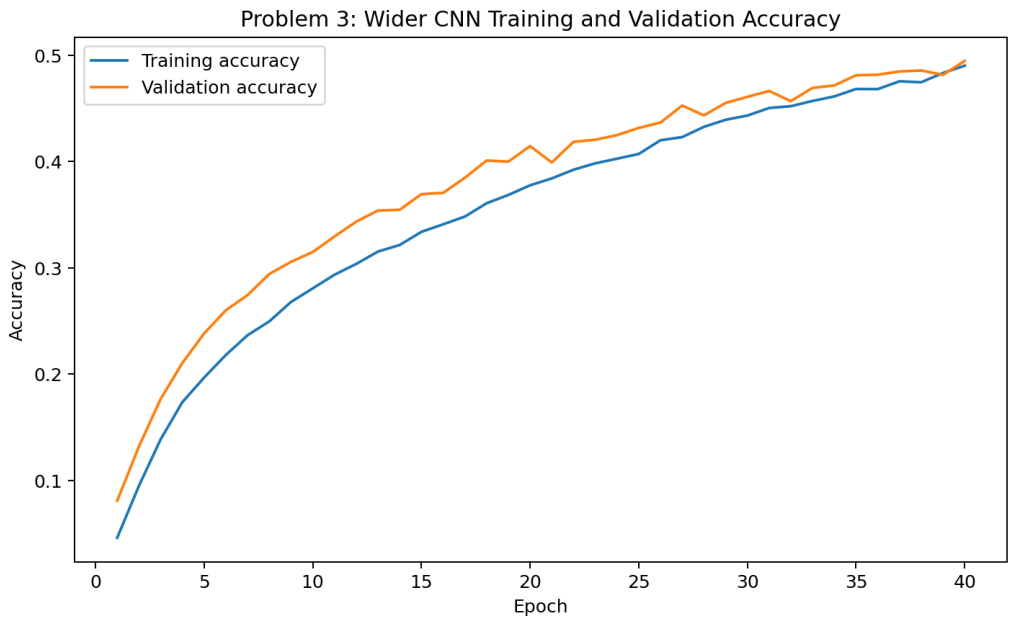

Figure 16: Training and validation accuracy for the wider augmented CNN.

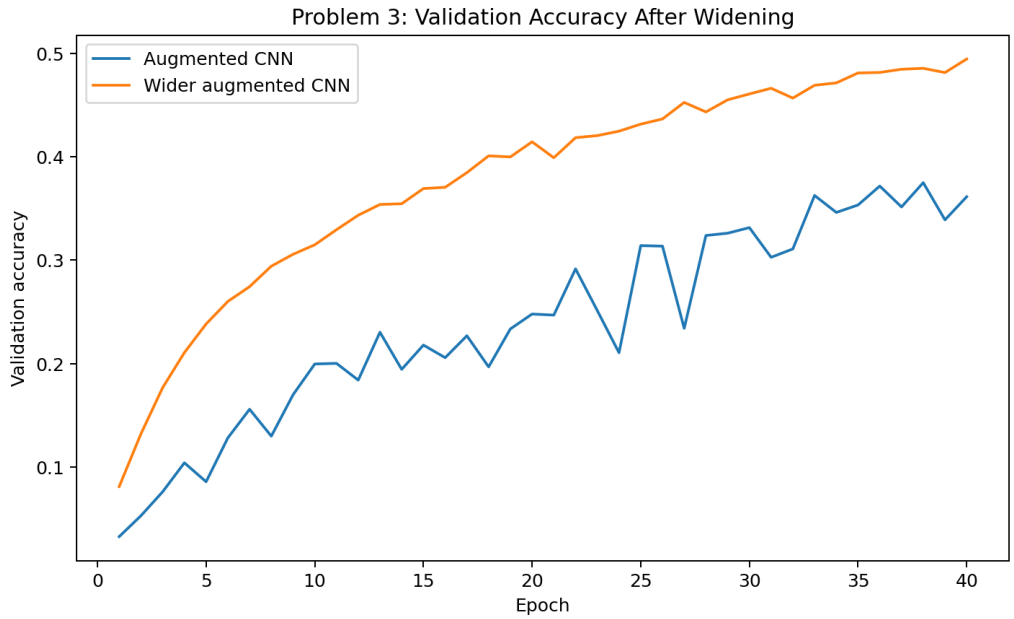

Figure 17: Validation accuracy for the augmented and wider CNNs.

Interpretation: Widening the network improved performance clearly. The wider augmented CNN reached 0.5025 test accuracy, or 0.4975 test error, compared with 0.3730 test accuracy for the narrower augmented CNN. This suggests the original augmented architecture was capacity-limited under this training setup.

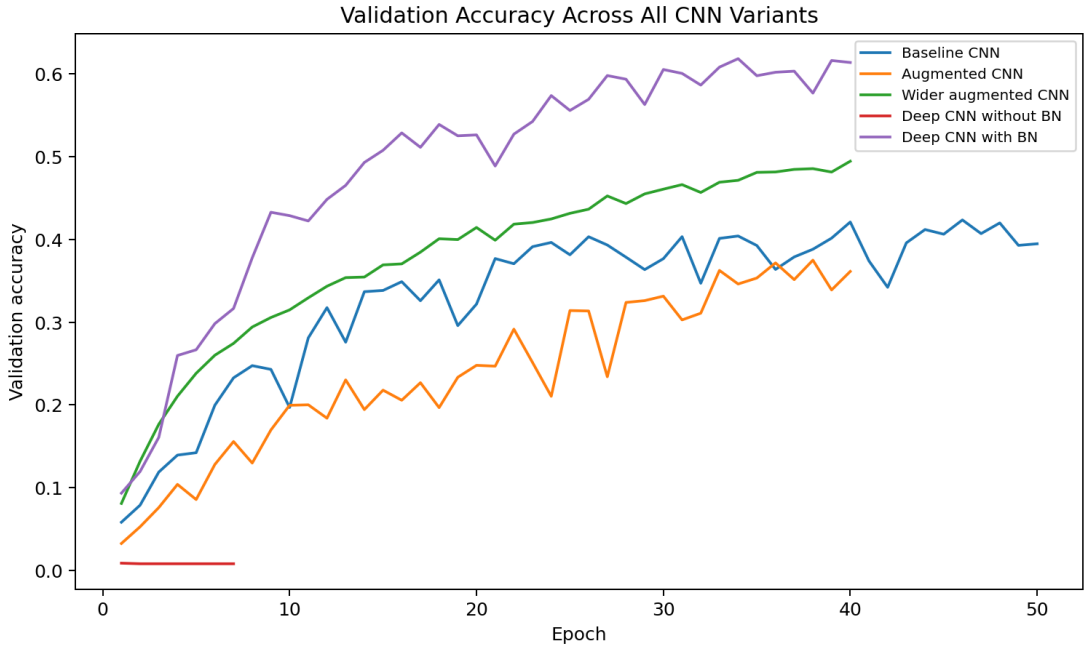

Figure 18: Validation accuracy across all trained CNN variants.

Model comparison: The all-model overlay shows the sequence of improvements more directly.

Augmentation alone is not enough, widening raises the ceiling, and the deep batch-normalized model separates from the other models after optimization stabilizes.

8 Problem 4: Deep Learning

8.1 4(a) Deeper CNN Without Batch Normalization

The deeper model stacks three convolutional layers before each max-pooling layer. This raises the number of trainable parameters to about 8.9 million.

The architecture complexity plot shows the jump from 4 convolutional layers in the wider network to 12 convolutional layers in the deeper network. The diagnostic for this section is a training/validation accuracy plot for the deep model without batch normalization.

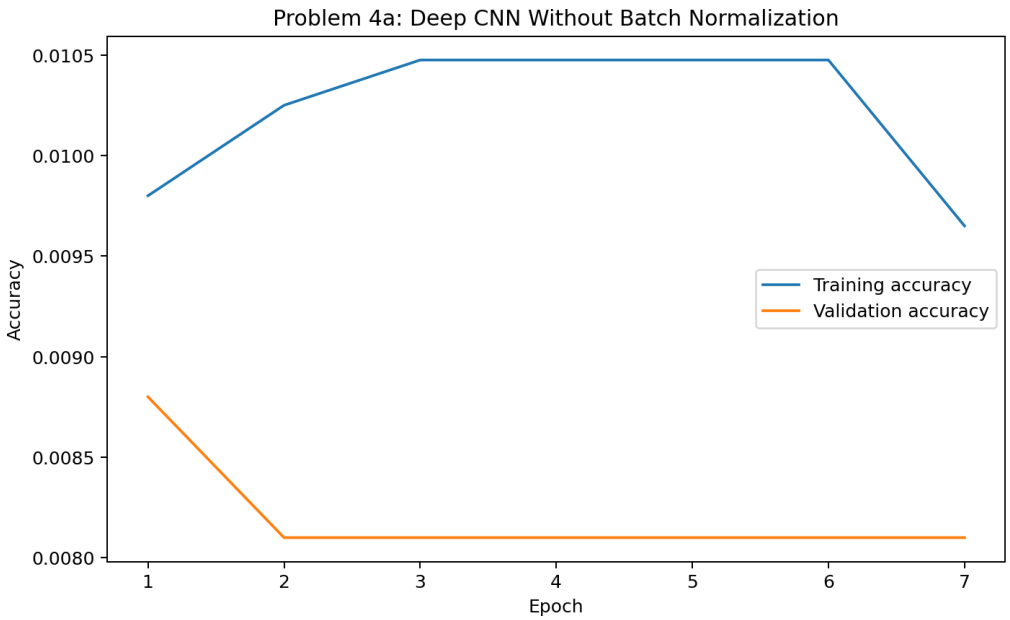

Figure 19: Training and validation accuracy for the deep CNN without batch normalization.

Interpretation: The deeper network without batch normalization failed to learn meaningful class structure. Its training accuracy stayed around 1%, and its test accuracy was 0.0100, which is essentially random guessing for 100 balanced classes. This supports the concern that deeper ReLU networks can become difficult to optimize without normalization.

8.2 4(b) Deeper CNN With Batch Normalization

The batch-normalized model uses the same deeper structure but normalizes activations before ReLU layers.

Batch normalization barely changes the parameter count, but it can strongly change optimization. The key figure for this part is a validation-accuracy overlay comparing the deeper network with and without batch normalization.

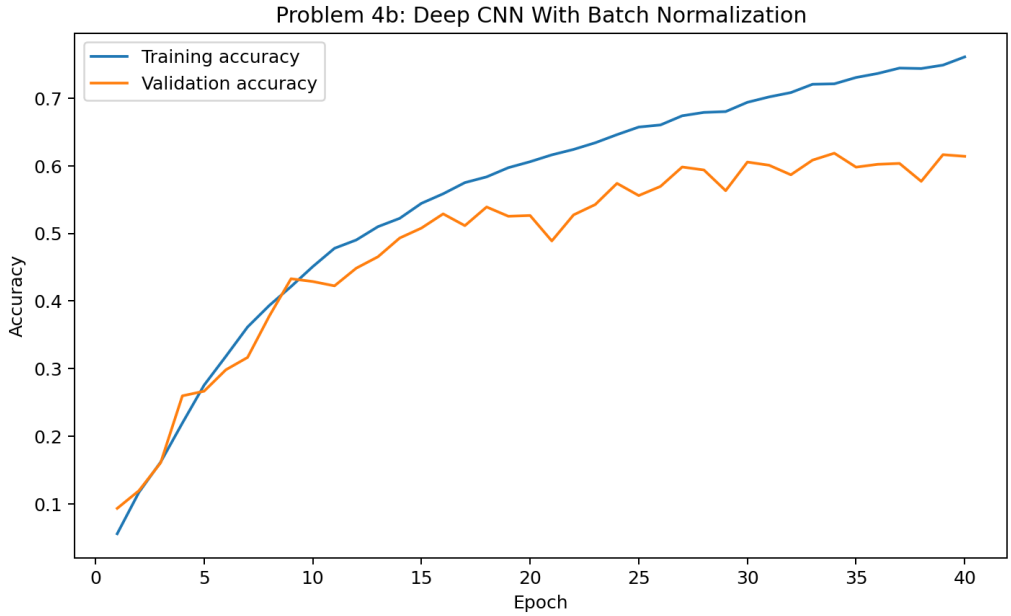

Figure 20: Training and validation accuracy for the deep CNN with batch normalization.

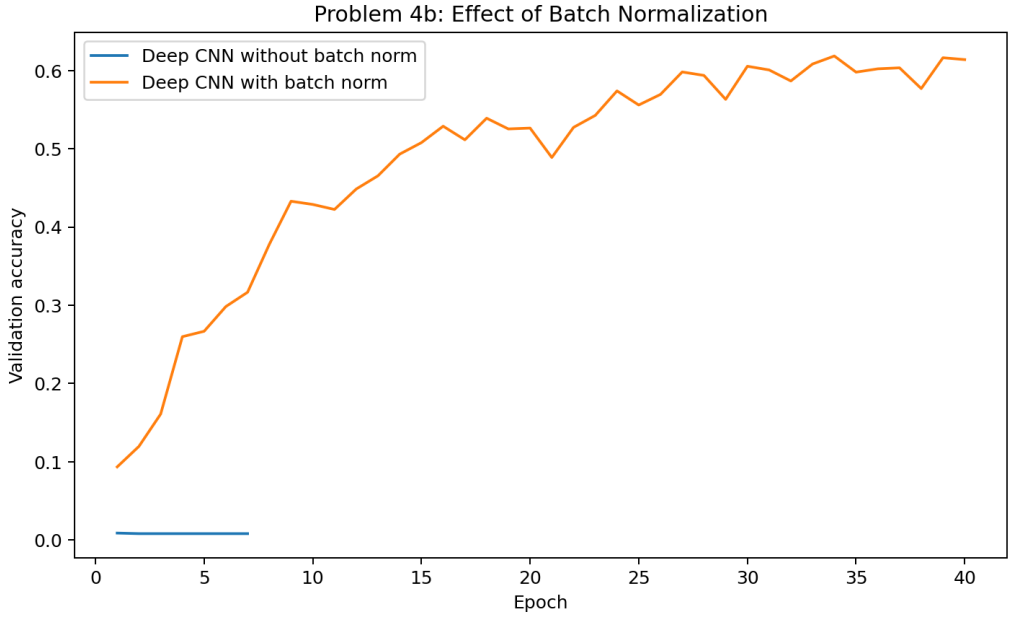

Figure 21: Validation accuracy for deep CNNs with and without batch normalization.

Interpretation: Batch normalization changed the deep model completely. With batch normalization, the deeper network reached 0.6177 test accuracy, or 0.3823 test error, compared with 0.0100

test accuracy without it. This is the strongest result in the assignment and shows why normalization is important for deeper CNNs.

8.3 4(c) Final Test Set Comparison

The final results are shown mainly as a bar chart of test accuracy by model. The exact values are also shown in the table below.

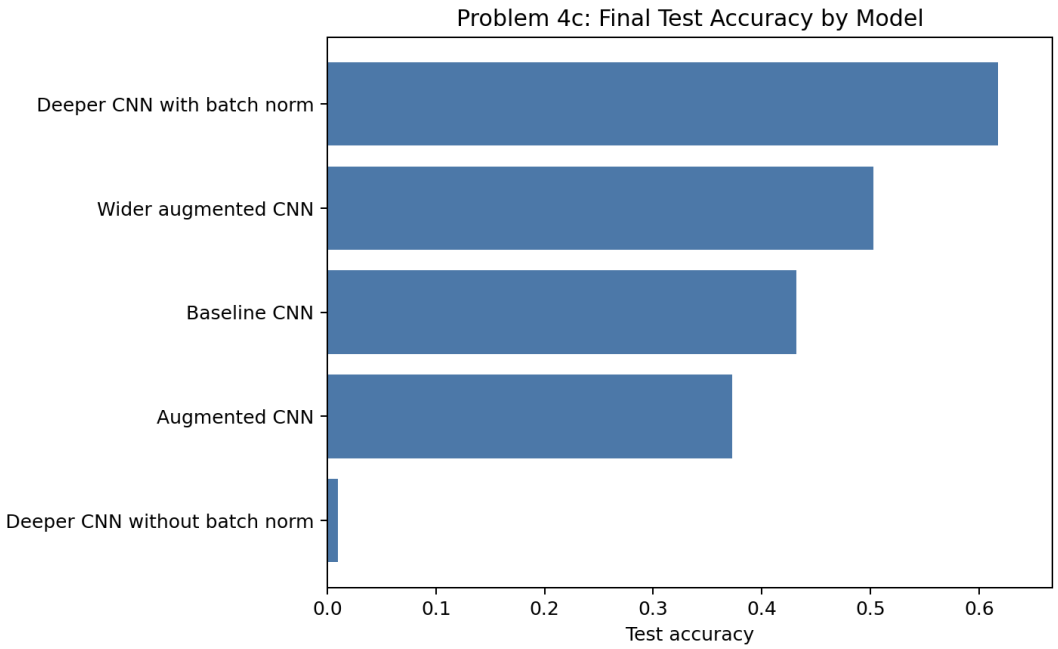

Figure 22: Final test accuracy comparison.

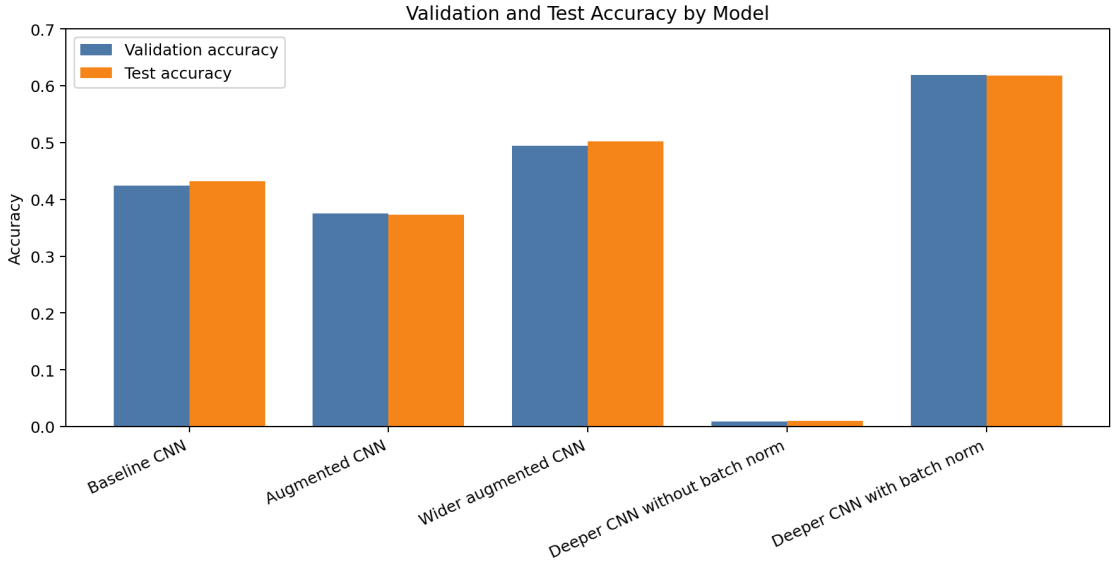

Figure 23: Validation and test accuracy comparison by model.

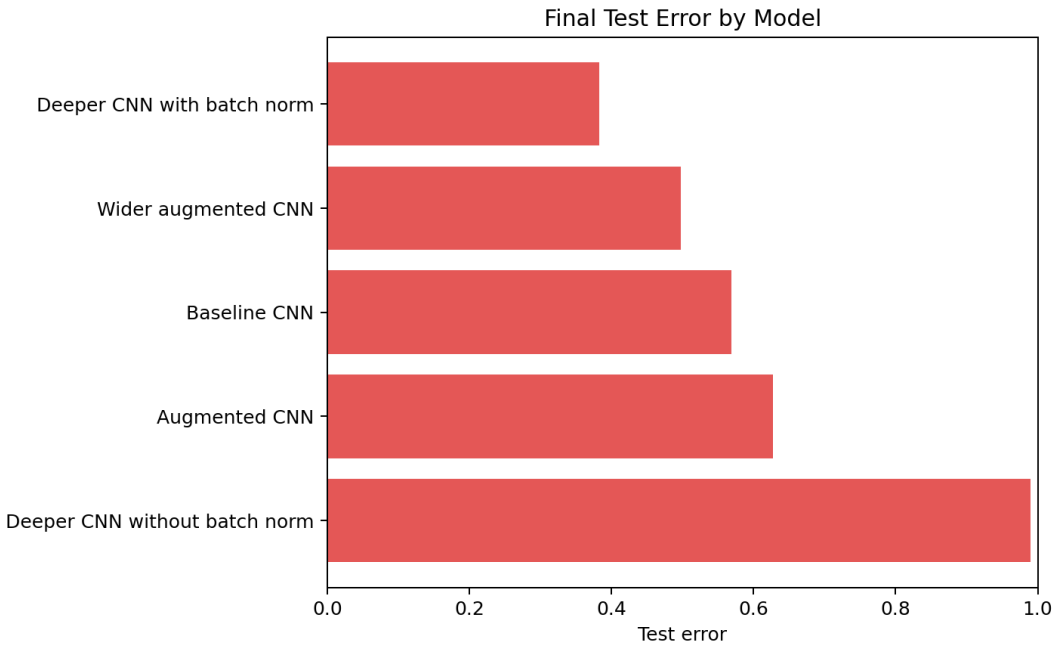

Figure 24: Final test error comparison by model.

Model	Best epoch	Validation accuracy	Test accuracy	Test error
Baseline CNN	46	0.4236	0.4316	0.5684
Augmented CNN	38	0.3750	0.3730	0.6270
Wider augmented CNN	40	0.4946	0.5025	0.4975
Deeper CNN without batch norm	1	0.0088	0.0100	0.9900
Deeper CNN with batch norm	34	0.6185	0.6177	0.3823

Interpretation: The best model was the deeper CNN with batch normalization, with 61.8% test accuracy. Widening helped, but depth only helped after batch normalization was added. The unnormalized deep model essentially failed, which is consistent with the vanishing-gradient/dead-neuron issue discussed in the lab prompt.

9 Final Summary

The baseline CNN provides the reference point. Data augmentation tests whether overfitting is limiting performance. Widening tests whether the model needs more filters. Deeper networks test whether additional layers help once optimization is stabilized by batch normalization.

The Colab run shows that architectural changes mattered more than augmentation alone in this shortened training schedule. The best result came from making the network deeper while adding batch normalization, which stabilized optimization and produced the strongest validation and test accuracy.

10 Reproducibility Note

The CNN training runs were executed in Google Colab using an A100 GPU runtime. The final report is prepared in Quarto, with the code included in the QMD source and hidden from the rendered PDF. The shareable Colab notebook link can be submitted with this report.